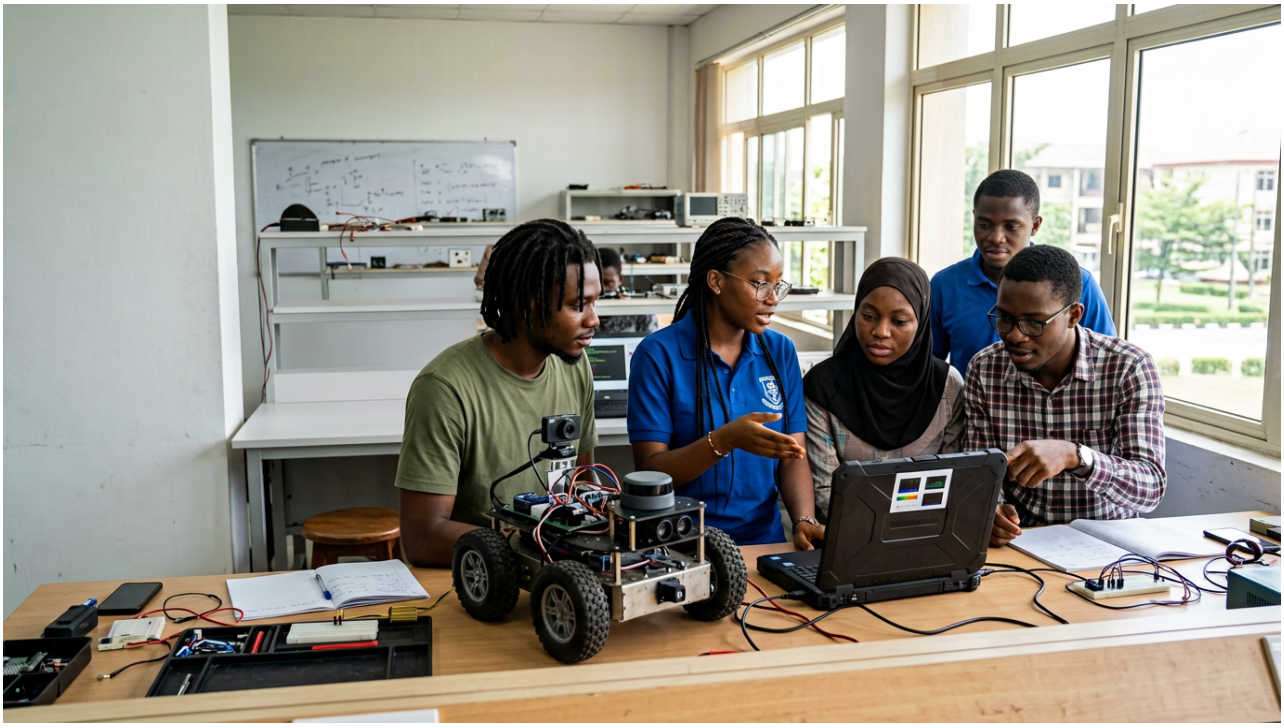
AI.101 — Introduction to Artificial Intelligence



Illustrative engineering-laboratory image; not documentary evidence of a specific university or event.

Narration manuscript · Module 7: Machine Learning Foundations

Academic level: Bachelor of Artificial Intelligence and Robotics Engineering
Content policy: No faith-related content.

1. Learning from experience

So far, we have examined agents that can use rules, search and structured plans. Another major approach is machine learning: methods that use data or experience to adjust a model so that it can perform a task. Learning is useful when writing every rule by hand is impractical, when patterns are too complex to specify directly, or when system behavior should improve with relevant examples.

Learning does not mean that a computer automatically understands a task. A learning algorithm receives a representation of examples and an objective. It adjusts parameters or structures to reduce an error measure or improve a reward. The engineer must choose what data to collect, how to represent it, which objective to optimize, how to evaluate generalization, and how to respond when the deployed environment differs from development data.

A machine-learning system is therefore a pipeline, not just a trained model. A typical pipeline includes defining the problem, collecting data, cleaning and labeling it, choosing a representation, splitting data for evaluation, training candidate models, comparing them with a baseline, testing on held-out examples, deploying the selected model, and monitoring performance. Mistakes early in the pipeline can invalidate later results.

2. Supervised learning

In **supervised learning**, the training examples include input data and a target label or value. The algorithm attempts to learn a mapping from inputs to targets. For a robot motor, inputs might include current draw, commanded speed, measured speed, ambient temperature and recent operating duration. The target might be motor temperature ten seconds later. If the target is a number, this is a regression task. If the target is a category such as normal, elevated or overheating, it is a classification task.

The input variables are often called **features**. The target can be called the **label**, response or target variable. The learned function is a model. The terms sound simple, but feature definition requires judgment. If a feature is unavailable at deployment time, it cannot be used reliably. If it contains information indirectly copied from the label, it may cause data leakage and make evaluation misleading.

A simple model can be a useful starting point. A linear regression estimates a weighted combination of features. A decision tree divides examples according to feature thresholds. More flexible models can represent complex relationships, but they may require more data, computation and validation. A more complicated model is not automatically better; it should outperform an appropriate baseline on a fair evaluation.

3. Unsupervised learning

In **unsupervised learning**, examples do not come with the target labels required for ordinary supervised learning. The algorithm looks for structure in the input data. Clustering groups examples according to a chosen notion of similarity. Dimensionality reduction maps high-dimensional data into a smaller representation that may be easier to inspect or use downstream. Anomaly detection identifies examples that differ from an expected pattern.

These methods can be valuable in robotics. A system might cluster vibration patterns from a motor to find recurring operating modes, or flag sensor readings that differ from historical behavior. But a cluster is not automatically a meaningful category. The result depends on how similarity is defined, which features are included, and how many groups are requested. Engineers must interpret and validate the discovered structure.

4. Reinforcement learning as a distinct setting

In **reinforcement learning**, an agent interacts with an environment and receives rewards or costs associated with outcomes. Unlike supervised learning, a complete correct action label is not necessarily provided for every situation. The agent must explore, observe consequences and learn a policy for selecting actions.

A robot learning to balance or navigate in simulation may receive a reward for staying upright or reaching a goal, and a penalty for collisions or excessive energy use. The reward function is a design choice. If we reward only rapid arrival, a robot may learn unsafe shortcuts. If simulation differs from the physical robot, a policy that succeeds in simulation may fail on hardware. Exploration can also be hazardous if performed in the real world without constraints. These issues make reinforcement learning a powerful but carefully governed method.

5. The purpose of training, validation and test data

To estimate whether a model generalizes beyond its examples, data are usually partitioned. A **training set** is used to fit the model. A **validation set** supports choices such as model complexity or hyperparameters. A **test set** is reserved for a final evaluation after development choices have been made.

The key principle is that the test set should act as an independent check. If a designer repeatedly examines test results and changes the system to improve them, the test set has become part of the development process. Performance on it may then be optimistic. For time-dependent data, a random split can also leak future information into training. A chronological split may be more appropriate: train on earlier time periods, validate on a later period, and test on the latest period.

Three separated datasets flow through training, model selection and final evaluation. Add a red crossed arrow showing that information from the test set must not flow back into tuning.]

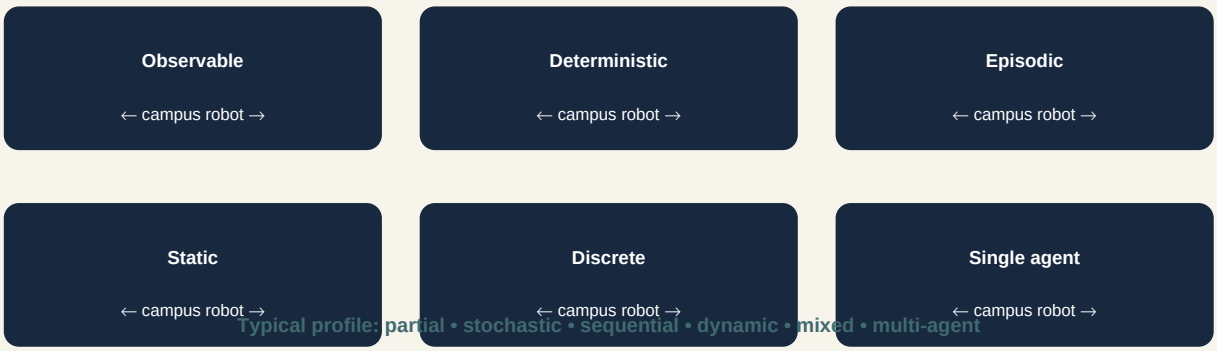


Diagram: Environment classification profile

6. Generalization and overfitting

A model **generalizes** when it performs well on new examples drawn from conditions relevant to its intended use. A model **overfits** when it captures details of the training examples that do not carry over to new cases. A highly flexible curve can pass through every training point yet oscillate wildly between them. On training data it may appear perfect, but on new observations its predictions can be poor.

Overfitting is one reason training accuracy alone is not enough. The gap between training and validation performance can indicate that a model has memorized patterns specific to the training set. Possible responses include collecting more representative data, reducing model complexity, using regularization, improving the feature representation, or changing the evaluation design. No single remedy is guaranteed; first investigate why the model fails.

Underfitting is the opposite problem: a model is too simple or poorly specified to capture important regularities even in the training examples. A linear model may not capture a strongly nonlinear relationship. The goal is not maximal complexity; it is an appropriate model that balances fit, robustness, interpretability, data requirements and operating cost.

7. A worked example: motor temperature prediction

Suppose a maintenance team wants to predict whether a motor will exceed a temperature limit. A first task definition should identify the prediction time horizon, the operating population, the cost of false alarms and missed overheating events, and what actions follow a prediction. “Predict motor temperature” is not yet complete if operators need to know whether they should slow the motor, inspect it, or stop it.

Potential features include current, speed, ambient temperature, recent load and time since startup. Labels may come from calibrated temperature sensors. Before training, inspect missing readings, units, sensor calibration, timestamps and class balance. If a temperature measurement taken ten seconds after the prediction moment is included as an input, the model has accidentally received information from the future. That is leakage.

Create a baseline, such as predicting the average future temperature or using a simple threshold rule. Then train a candidate model and compare it on a time-based test split. For regression, metrics might include mean absolute error and root mean squared error. For classification, precision, recall, confusion matrices and calibration may matter. If missing an overheating event is much worse than issuing a false alarm, overall accuracy is not a sufficient metric.

Finally, consider deployment. Sensors may drift. A replacement motor may have different characteristics. Ambient temperatures may change seasonally. The model should be monitored, and there should be a safe fallback if inputs are missing or outside the training range. A model score is not the complete maintenance procedure.

8. Evaluation without self-deception

A fair model comparison uses the same data partitions, preprocessing rules and metrics for all candidates. If scaling or feature selection is learned from the data, fit it only on the training portion and apply the learned transformation to validation and test data. Otherwise, information from the held-out data can leak into training indirectly.

Use cross-validation when appropriate to estimate variability across data splits, but respect groups and time structure. Multiple measurements from the same physical motor should not be split across training and test if the goal is to generalize to unseen motors; otherwise, the model may recognize motor-specific signatures. The evaluation split should match the intended deployment question.

Always compare with a baseline. If a sophisticated model barely improves on a simple rule, its additional computation and maintenance may not be justified. If it performs better on an average metric but fails on a critical subgroup, the improvement may be unacceptable. Report uncertainty and limitations alongside the headline score.

9. Practical lab: build a leakage-aware classifier

Use a small public or instructor-prepared dataset with a clear target. In Python, inspect the columns, missing values, units, class balance and potential leakage fields. Create a train/test split suited to the data structure. Fit a simple baseline first. Train at least two models, such as logistic regression and a decision tree, using a validation procedure. Evaluate with a confusion matrix and metrics chosen for the use case.

Plot the data and compare predicted probabilities with outcomes. Identify false positives and false negatives, then inspect representative cases. Do not use protected or sensitive attributes casually; consider whether they are necessary, whether they introduce harm, and how evaluation should test performance differences. For an engineering dataset, identify whether the train/test split separates devices, sites, time periods or users as needed.

Submit a short experiment report with the task statement, data source, split method, baseline, model choices, metric rationale, results, failure analysis and limitations. Include the exact procedure needed to reproduce the

result. A model that cannot be reproduced or explained at a basic engineering level is difficult to maintain.

10. Common mistakes

A common mistake is to begin by choosing a popular model instead of defining the task. Another is to trust a high training score. A third is to split data randomly when records are linked in time, by person or by device. A fourth is to include features unavailable when the model is used. A fifth is to report only overall accuracy when rare but consequential failures matter. A final mistake is to assume that a learned correlation is causal or stable under changing conditions.

11. Knowledge check and assignment

Explain the distinction between supervised, unsupervised and reinforcement learning. What separates training, validation and test data? Give an example of data leakage in a sensor prediction task. Why can a model achieve high training accuracy and still fail after deployment? What baseline could be appropriate for motor-temperature prediction? Which evaluation measure would you select if missed overheating is much more costly than a false alarm, and why?

For the assignment, complete an end-to-end experiment on a modest dataset. State the deployment question, implement a baseline and at least two candidate methods, use a defensible split, select metrics that reflect consequences, and report limitations. Include a simple monitoring plan for data drift and sensor failure.

Module summary

Machine learning adjusts models from data or interaction, but it does not eliminate engineering judgment. We introduced supervised, unsupervised and reinforcement learning; features, labels and models; data partitioning; generalization, overfitting and underfitting; and evaluation against baselines. We emphasized leakage prevention, deployment monitoring and consequence-aware metrics. In the next module, we will examine how neural networks learn representations and why their capabilities must be assessed alongside their limitations.